МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение высшего образования

**«Российский экономический университет имени Г.В. Плеханова»**

Пермский институт (филиал)

# Отчет

по выполнению индивидуального задания

по дисциплине «Технология разработки программного обеспечения»

по теме: «Создание подсистемы для работы с партнерами компании»

Работу выполнил
студент группы: ИПо-42
А.Е. Гуляев

«16» Марта 2026 г.

Пермь, 2026 г.

# Оглавление

**Введение** ..... 2
**ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ** ..... 3
**ГЛАВА 2. СОЗДАНИЕ БАЗЫ ДАННЫХ** ..... 6
**2.1. Создание роли app с паролем** ..... 6
**2.2 Создание базы данных** ..... 6
**2.3 Подключение к БД и создание схемы app** ..... 6
**2.4 Установка схемы app по умолчанию для роли app** ..... 7
**2.5 Таблицы и их структура** ..... 7
**2.6. (Опционально) Наполнение данными** ..... 8
**2.7. Права доступа** ..... 9
**2.8. Резервное копирование базы данных** ..... 9
**2.9. Полный SQL-скрипт развёртывания базы данных** ..... 10
**2.10.Схема базы данных (DbSchema)** ..... 11
**ГЛАВА 3. РАЗРАБОТКА ПРИЛОЖЕНИЯ** ..... 13
**3.1. Элементы интерфейса** ..... 13
**ГЛАВА 4. ТЕСТИРОВАНИЕ БИБЛИОТЕКИ** ..... 16
**Заключение** ..... 20
**ПРИЛОЖЕНИЯ** ..... 22

# Введение

В рамках выполнения практического задания была разработана программа «Учёт партнёров и реализации продукции», предназначенная для хранения и обработки информации о партнёрах, типах партнёров, номенклатуре продукции и истории её реализации. Основной целью разработки является создание структурированной базы данных, обеспечивающей удобное хранение информации, быстрый поиск данных и выполнение необходимых расчетов.

В процессе разработки были выполнены следующие этапы:

1. анализ предметной области и формирование требований к программному обеспечению;
2. проектирование логической структуры базы данных, включая выделение сущностей (партнёр, тип партнёра, продукт, история реализации продукции) и связей между ними;
3. физическая реализация базы данных, наполнение её тестовыми данными;
4. разработка приложения с графическим интерфейсом для взаимодействия с базой данных (платформа .NET, технология WPF);
5. реализация сервисного слоя для выполнения операций добавления, редактирования, удаления и расчётов;
6. отладка и тестирование программных модулей;
7. инспектирование кода на соответствие стандартам кодирования.

Разработка базы данных осуществлялась с использованием системы управления базами данных PostgreSQL. Для визуального проектирования структуры базы данных применялось средство моделирования DbSchema, которое позволило наглядно представить таблицы, их поля и взаимосвязи, а также сгенерировать SQL-скрипты для создания схемы. Клиентская часть приложения разработана в среде Microsoft Visual Studio 2022 на языке C# с применением объектно-реляционного отображения (Entity Framework Core) для взаимодействия с базой данных

# ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Для разработки системы необходимо проанализировать, какие данные должны предоставляться пользователю и какими он может манипулировать. Полное описание предметной области представлено в приложении 1.

В рамках данного задания необходимо разработать подсистему для работы с партнерами компании, обеспечивающую следующий функционал:

- просмотр списка партнеров;
- добавление/редактирование данных о партнере;
- просмотр истории реализации продукции партнером.

При отображении истории реализации продукции должна выводиться следующая информация:

- наименование продукции;
- количество реализованной продукции;
- дата продажи.

При отображении и создании партнера должна использоваться следующая информация:

- наименование;
- тип партнера;
- рейтинг;
- адрес;
- ФИО директора;
- телефон;
- электронная почта.

Дополнительно в рамках задания было предоставлено следующее требование: наличие возможности добавления/редактирования/удаления не только партнеров, но связанных с ними сущностей (тип партнера, продукция, история реализации продукции).

Исходя из предоставленных данных, для реализации системы необходимо в базе данных отобразить только четыре сущности:

- продукция;

- типы партнеров;
- партнеры;
- история реализации продукции партнерами.

Продукция представляет собой объект, у которого есть только артикул, который используется как уникальное значение для разграничения продукции между собой, и наименование. Так как в требованиях не указаны пункты о выводе другой информации, такой как стоимость продукции, изображение и прочее, то она не будет усмотрена в данной работе.

Тип партнера представляет собой объект, содержащий уникальное название и не более.

Партнер – комплексный объект, который кроме ранней представленной информации содержит и идентификатор своего типа.

История реализации продукции – объект, хранящий информацию о реализованной продукции, партнере, который реализовал продукцию, а также количество и дату продажи данной продукции.

В ходе анализа предметной области были выявлены основные сущности базы данных и их содержание, что позволяет осуществить создание базы данных для разрабатываемой системы.

# ГЛАВА 2. СОЗДАНИЕ БАЗЫ ДАННЫХ

## 2.1. Создание роли app с паролем

```
Microsoft Windows [Version 10.0.22631.6199]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\aegis>cd "C:\Program Files\PostgreSQL\17\bin"

C:\Program Files\PostgreSQL\17\bin>psql -U postgres
psql (17rc1)
ПРЕДУПРЕЖДЕНИЕ: Кодовая страница консоли (866) отличается от основной
                страницы Windows (1251).
                8-битовые (русские) символы могут отображаться некорректно.
                Подробнее об этом смотрите документацию psql, раздел
                "Notes for Windows users".
Введите "help", чтобы получить справку.

postgres=# CREATE ROLE app WITH LOGIN PASSWORD '123';
ОШИБКА:  роль "app" уже существует
postgres=#
```

Рисунок 1 – роль app, а также пароль.

## 2.2 Создание базы данных

```
postgres=# CREATE DATABASE gulyaev_db OWNER app ENCODING 'UTF8';
CREATE DATABASE
postgres=#
```

Рисунок 2 – Создание базы данных

## 2.3 Подключение к БД и создание схемы app

```
postgres=# \c gulyaev_db
Вы подключены к базе данных "gulyaev_db" как пользователь "postgres".
gulyaev_db=#
```

Рисунок 3 – Подключение к БД и создание схемы app.

## 2.4 Установка схемы app по умолчанию для роли app

```
gulyaev_db=# CREATE SCHEMA app AUTHORIZATION app;
CREATE SCHEMA
gulyaev_db=# ALTER ROLE app SET search_path TO app;
ALTER ROLE
gulyaev_db=# |
```

Рисунок 4 – Установка схемы app по умолчанию для роли app.

## 2.5 Таблицы и их структура

Таблица partners

```
CREATE SCHEMA
gulyaev_db=# ALTER ROLE app SET search_path TO app;
ALTER ROLE
gulyaev_db=# CREATE TABLE app.partners (
gulyaev_db(#   id SERIAL PRIMARY KEY,
gulyaev_db(#   partner_type VARCHAR(5),
gulyaev_db(#   partner_name VARCHAR(50),
gulyaev_db(#   partner_director VARCHAR(120),
gulyaev_db(#   partner_email VARCHAR(120),
gulyaev_db(#   partner_number VARCHAR(12),
gulyaev_db(#   partner_address VARCHAR(240),
gulyaev_db(#   partner_inn VARCHAR,
gulyaev_db(#   partner_rating SMALLINT
gulyaev_db(# );
CREATE TABLE
gulyaev_db=# |
```

Рисунок 5 – Таблица partners.

Таблица product_type

```
CREATE TABLE
gulyaev_db=# CREATE TABLE app.product_type (
gulyaev_db(#   id SERIAL PRIMARY KEY,
gulyaev_db(#   product_type VARCHAR(20),
gulyaev_db(#   coaf_product_type REAL
gulyaev_db(# );|
```

Рисунок 6 – Таблица product_type.

Таблица products

```
CREATE TABLE
gulyaev_db(#   id SERIAL PRIMARY KEY,
gulyaev_db(#   product_type INTEGER REFERENCES app.product_type(id),
gulyaev_db(#   product_name VARCHAR(120),
gulyaev_db(#   partner_artc INTEGER,
gulyaev_db(#   partner_price REAL
gulyaev_db(# );|
```

Рисунок 7 – Таблица products.

Таблица partner_products

```
CREATE TABLE
gulyaev_db(#   id SERIAL PRIMARY KEY,
gulyaev_db(#   product INTEGER REFERENCES app.products(id),
gulyaev_db(#   partner INTEGER REFERENCES app.partners(id),
gulyaev_db(#   partner_count INTEGER,
gulyaev_db(#   sale_date DATE
gulyaev_db(# );
```

Рисунок 8 – Таблица partner_products.

## 2.6. (Опционально) Наполнение данными

```
gulyaev_db(#   sale_date DATE
gulyaev_db(# );INSERT INTO app.product_type (product_type, coaf_product_type) VALUE
CREATE TABLE
gulyaev_db-# ('Ламинат', 2.35),
gulyaev_db-# ('Массивная доска', 5.15),
gulyaev_db-# ('Паркетная доска', 4.34),
gulyaev_db-# ('Пробковое покрытие', 1.5);
```

Рисунок 9 – Наполнение данными

```
INSERT 0 4
gulyaev_db=# INSERT INTO app.products (product_type, product_name, partner_artc, partner_price) VALUES
gulyaev_db-# (3, 'Паркетная доска Ясень темный однополосная 14 мм', 8758385, 4456.90),
gulyaev_db-# (3, 'Инженерная доска Дуб Французская елка однополосная 12 мм', 8858958, 7330.99),
gulyaev_db-# (1, 'Ламинат Дуб дымчато-белый 33 класс 12 мм', 7750282, 1799.33),
gulyaev_db-# (1, 'Ламинат Дуб серый 32 класс 8 мм с фаской', 7028748, 3890.41),
gulyaev_db-# (4, 'Пробковое напольное клеевое покрытие 32 класс 4 мм', 5012543, 5450.59);
```

Рисунок 10 – Наполнение данными

Описание таблиц базы данных:

˜ app.partners – содержит данные о партнёрах (контрагентах): тип партнёра, наименование компании, ФИО директора, контактную информацию (email, телефон), адрес, ИНН и рейтинг.

˜ app.product_type – справочник типов выпускаемой продукции (например, ламинат, паркетная доска) с дополнительным коэффициентом (вероятно, для расчётов).

˜ app.products – каталог продукции: ссылка на тип продукта, название, артикул партнёра и цена для партнёра.

˜ app.partner_products – журнал продаж, связывающий партнёров и продукцию: указывается, какой продукт, какому партнёру, в каком количестве и когда был реализован.

В приведённых SQL-скриптах также показано начальное наполнение справочников типа продукции и нескольких товаров.

### 2.7. Права доступа

Для обеспечения безопасности базы данных и разграничения доступа пользователей была использована ролевая модель управления доступом.

В системе используется роль app, которая предназначена для работы приложения с базой данных. Данной роли назначены права на выполнение основных операций с объектами схемы app.

Основные действия:

- создание роли пользователя;
- назначение пароля;
- установка схемы app в качестве схемы по умолчанию;
- предоставление прав доступа к таблицам.

Использование отдельной роли для работы приложения позволяет повысить уровень безопасности и обеспечить контроль доступа к данным.

### 2.8. Резервное копирование базы данных

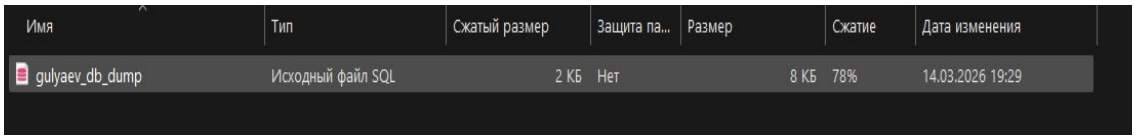
Для обеспечения сохранности данных было выполнено резервное копирование базы данных с использованием утилиты pg_dump, входящей в состав системы управления базами данных PostgreSQL.

Резервная копия базы данных позволяет восстановить информацию в случае сбоя системы, повреждения данных или других непредвиденных ситуаций.

Команда резервного копирования базы данных:

pg_dump -U postgres -d gulyaev_db -Fc > gulyaev_db_backup.dump

В результате выполнения команды создаётся файл резервной копии базы данных, который может быть использован для последующего восстановления данных.



| Имя | Тип | Сжатый размер | Защита па... | Размер | Сжатие | Дата изменения |
|---|---|---|---|---|---|---|
| gulyaev_db_dump | Исходный файл SQL | 2 КБ | Нет | 8 КБ | 78% | 14.03.2026 19:29 |

Рисунок 11 – Создание резервной копии базы данных.

### 2.9. Полный SQL-скрипт развёртывания базы данных

Для удобства повторного развёртывания базы данных был подготовлен SQL-скрипт, содержащий полный набор команд для создания структуры базы данных.

Предоставленный SQL-скрипт содержит:

˜Создание таблиц**:** partners, product_type, products, partner_products с указанием полей и типов данных.

˜Определение первичных ключей**:** для каждой таблицы задан PRIMARY KEY через SERIAL.

˜Создание связей (внешних ключей)**:** в таблицах products и partner_products использовано REFERENCES, что обеспечивает ссылочную целостность между таблицами.

˜Частичное заполнение тестовыми данными**:** вставлены записи в таблицы product_type (4 типа) и products (5 товаров).

Использование единого SQL-скрипта позволяет быстро развернуть структуру базы данных на новом сервере или восстановить систему после переустановки.

```sql
CREATE TABLE app.partners (
  id SERIAL PRIMARY KEY,
  partner_type VARCHAR(5),
  partner_name VARCHAR(50),
  partner_director VARCHAR(120),
  partner_email VARCHAR(120),
  partner_number VARCHAR(12),
  partner_address VARCHAR(240),
  partner_inn VARCHAR,
  partner_rating SMALLINT
);

CREATE TABLE app.product_type (
  id SERIAL PRIMARY KEY,
  product_type VARCHAR(20),
  coaf_product_type REAL
);

CREATE TABLE app.products (
  id SERIAL PRIMARY KEY,
  product_type INTEGER REFERENCES app.product_type(id),
  product_name VARCHAR(120),
  partner_artc INTEGER,
  partner_price REAL
);

CREATE TABLE app.partner_products (
  id SERIAL PRIMARY KEY,
  product INTEGER REFERENCES app.products(id),
  partner INTEGER REFERENCES app.partners(id),
  partner_count INTEGER,
  sale_date DATE
);

INSERT INTO app.product_type (product_type, coaf_product_type) VALUES
('Ламинат', 2.35),
('Массивная доска', 5.15),
('Паркетная доска', 4.34),
('Пробковое покрытие', 1.5);

INSERT INTO app.products (product_type, product_name, partner_artc, partner_price) VALUES
(3, 'Паркетная доска Ясень темный однополосная 14 мм', 8758385, 4456.90),
(3, 'Инженерная доска Дуб Французская елка однополосная 12 мм', 8858958, 7330.99),
(1, 'Ламинат Дуб дымчато-белый 33 класс 12 мм', 7750282, 1799.33),
(1, 'Ламинат Дуб серый 32 класс 8 мм с фаской', 7028748, 3890.41),
(4, 'Пробковое напольное клеевое покрытие 32 класс 4 мм', 5012543, 5450.59);
```

Рисунок 12 – Полный SQL-скрипт базы данных.

### 2.10.Схема базы данных (DbSchema)

Для визуального представления структуры базы данных была построена схема базы данных с использованием специализированного инструмента моделирования DbSchema. Схема базы данных отражает:

1. перечень таблиц: partners, product_type, products, partner_products;

2. атрибутный состав каждой таблицы (наименования полей, типы данных, ограничения);

3. первичные ключи (PK), обеспечивающие уникальную идентификацию записей;

4. внешние ключи (FK), определяющие логические связи между таблицами и поддерживающие ссылочную целостность;

5. индексы (при их наличии), оптимизирующие выполнение запросов.

Визуализация позволяет наглядно оценить структуру данных, проверить корректность связей и облегчает дальнейшее сопровождение базы данных.

Использование графической схемы позволяет наглядно представить структуру базы данных и связи между её объектами.

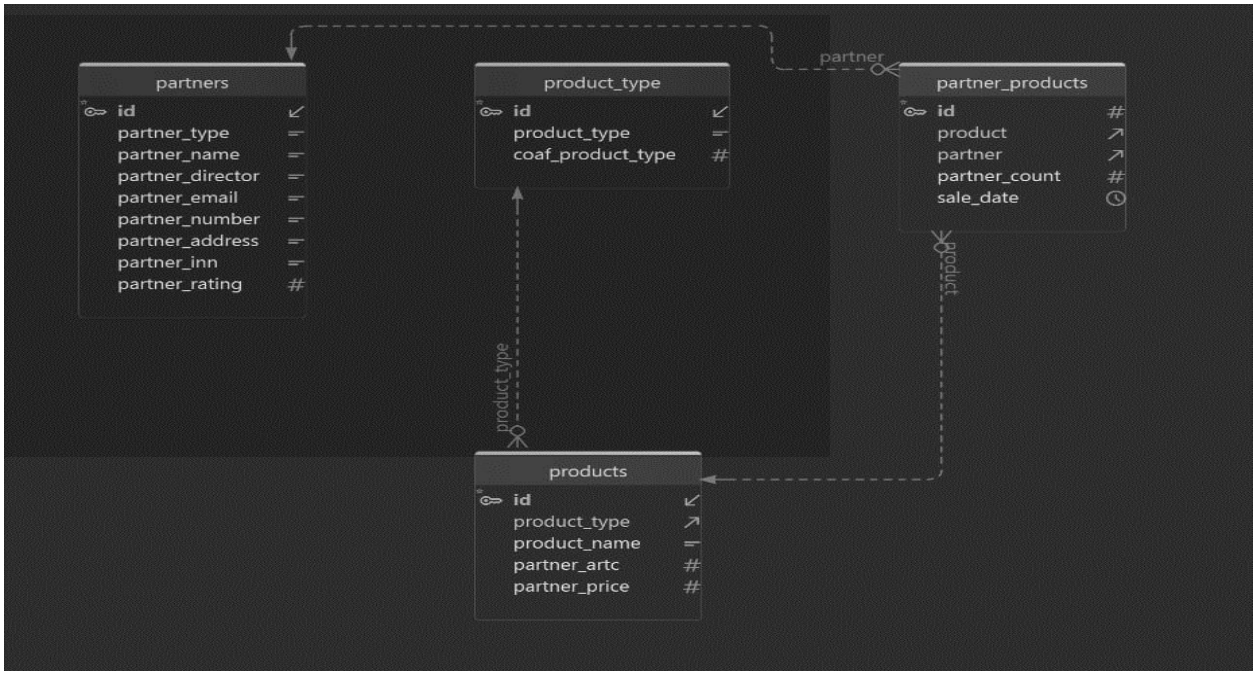



Рисунок 13 – Схема базы данных.

# ГЛАВА 3. РАЗРАБОТКА ПРИЛОЖЕНИЯ

## 3.1. Элементы интерфейса

Главное окно приложения (рисунок 7) содержит два основных блока. В верхней части отображается список всех партнёров в виде карточек. По каждой записи выводится краткая информация: тип партнёра и название организации, контактные данные, рейтинг, а также автоматически рассчитанный процент скидки (зависит от общего объёма продаж). В нижней части окна расположена таблица с историей реализации продукции, где отражены наименование товара, количество и дата продажи. Навигация осуществляется с помощью кнопок добавления, редактирования и удаления записей.

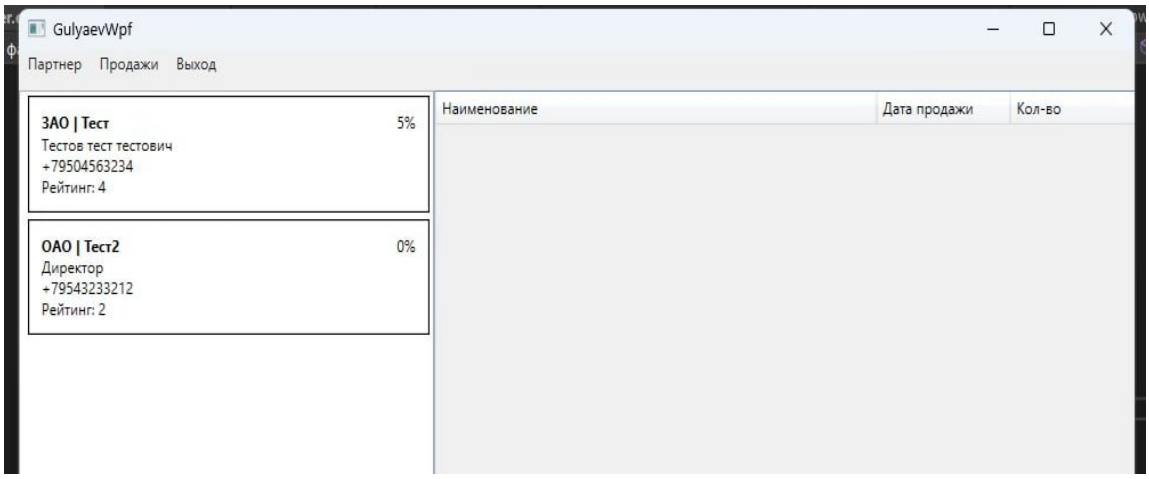



Рисунок 14 – Главное окно приложения.

Окно редактирования партнёра содержит поля для изменения данных партнёра. Обязательные поля визуально выделены, тип партнёра выбирается из выпадающего списка.

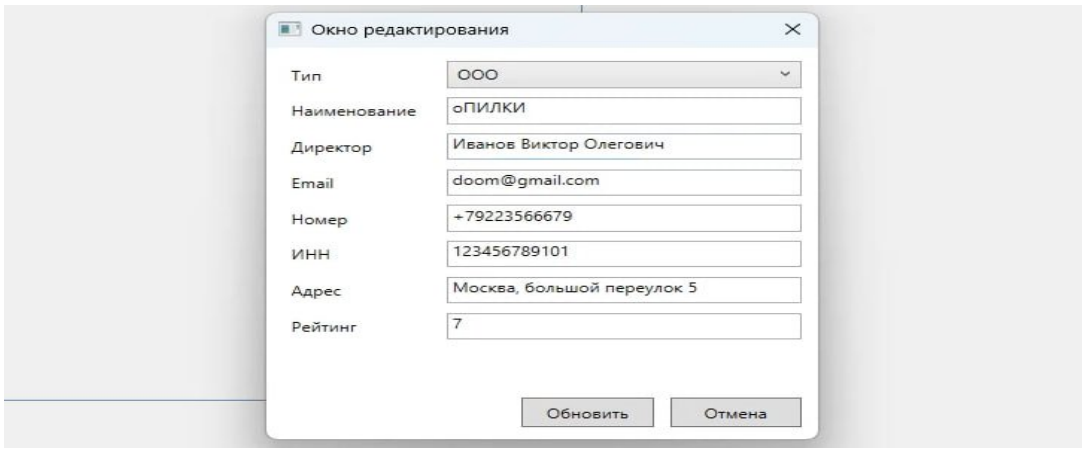



Рисунок 15 – Окно редактирования партнёра.

Окно добавления продукции предназначено для внесения новой записи о реализации товара. Интерфейс содержит поле выбора продукта из выпадающего списка, календарь для указания даты продажи и числовое поле для ввода количества. В нижней части окна расположены кнопки «Добавить» для сохранения записи и «Отмена» для выхода без сохранения.

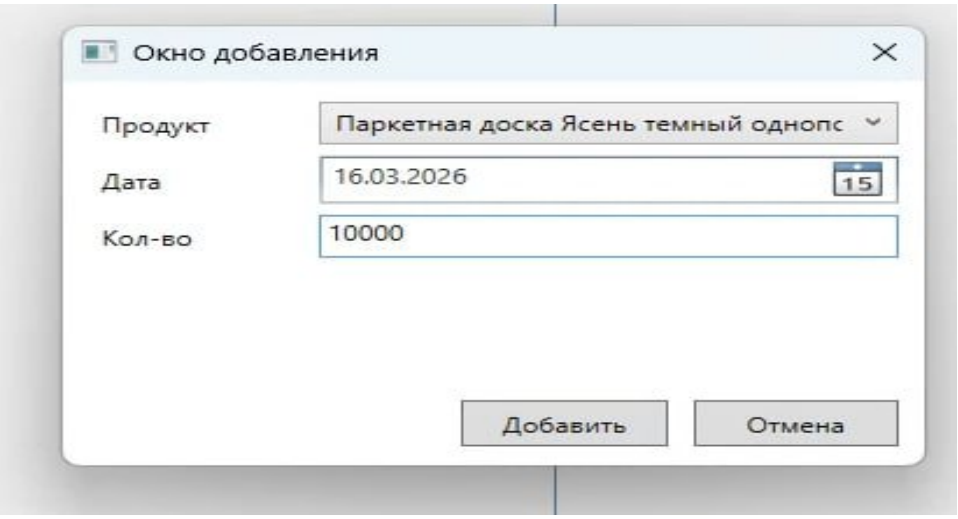



Рисунок 16 – Окно добавления продукции

Окно подтверждения удаления (рисунок 10) представляет собой стандартный диалог с запросом подтверждения действия. Содержит текстовый вопрос «Вы уверены, что хотите удалить партнёра?» и две кнопки: «Да» для выполнения удаления и «Нет» для отмены операции.

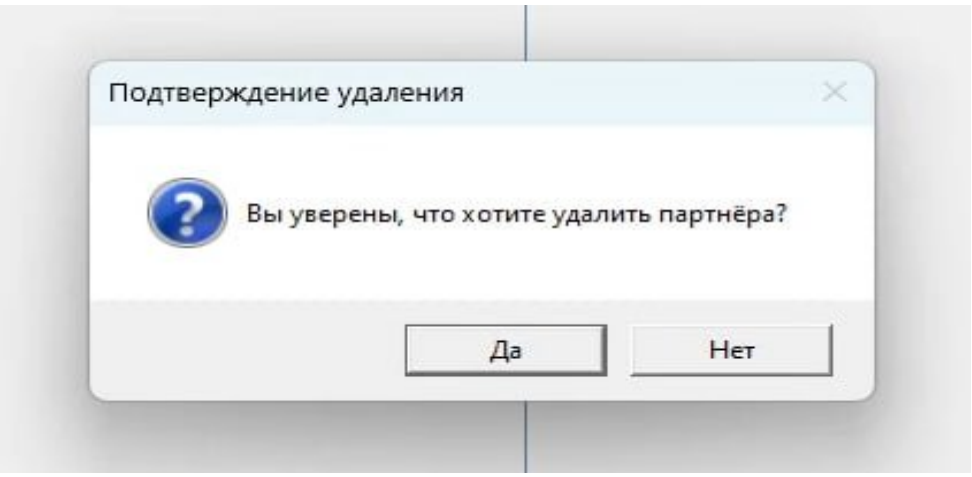



Рисунок 17 – Окно подтверждения удаления партнёра

Диаграмма классов отражает статическую структуру программной системы: основные классы проекта, их атрибуты (свойства) и взаимосвязи между ними. На ней представлены классы-сущности, соответствующие таблицам базы данных (Partner, ProductType, Product, PartnerProduct), класс контекста (ApplicationContext), а также сервисный класс (Service), содержащий бизнес-логику приложения. Связи между классами показывают направления навигации (один ко многим, многие к одному) и отражают архитектуру, реализованную с использованием Entity Framework Core. Диаграмма даёт наглядное представление о ключевых компонентах программного модуля и служит основой для понимания его устройства и дальнейшего развития.

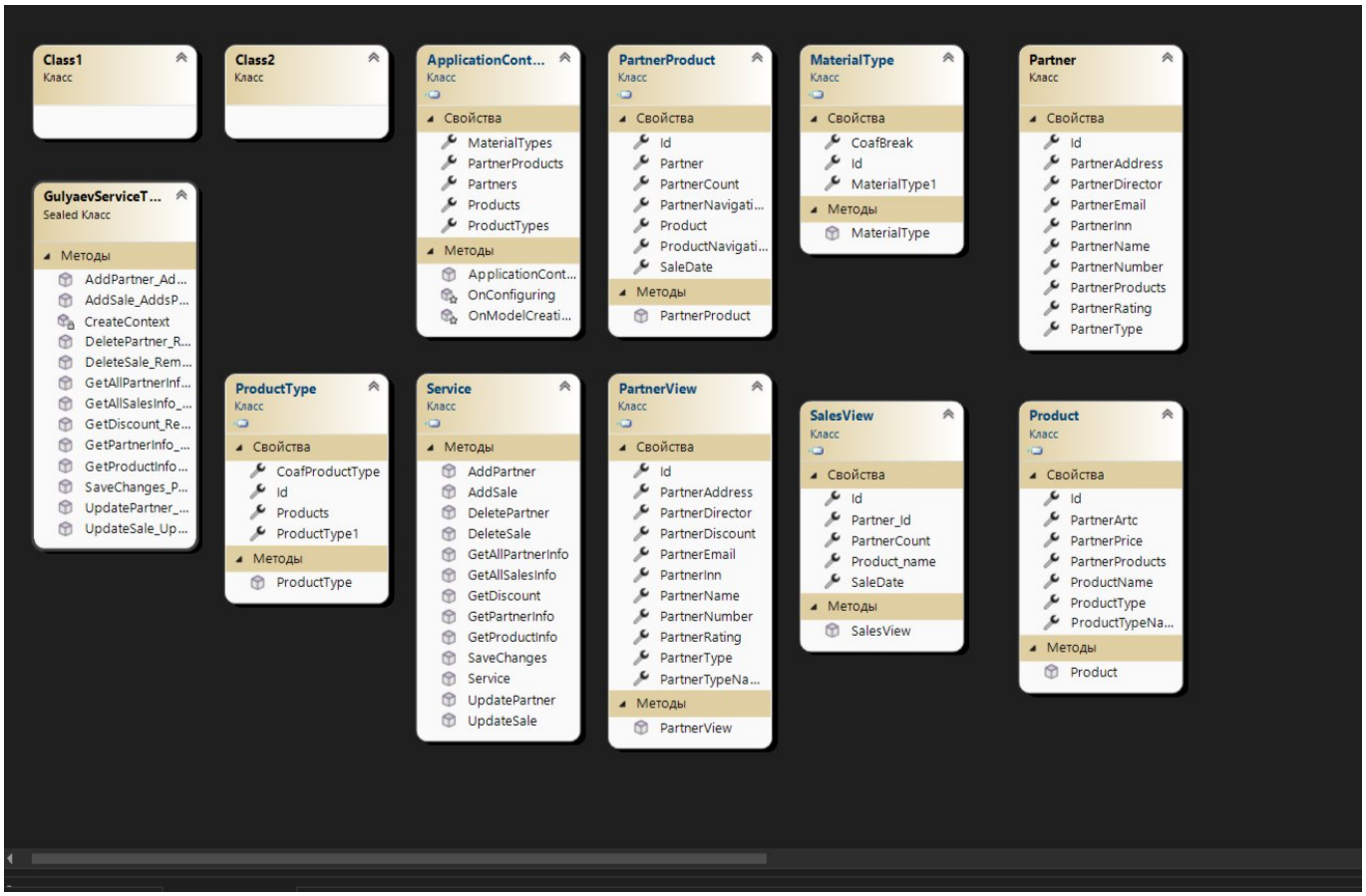



Рисунок 18 – Диаграмма классов

# ГЛАВА 4. ТЕСТИРОВАНИЕ БИБЛИОТЕКИ

В процессе разработки библиотеки классов GulyaevLib, содержащей логику работы с данными партнёров, продукции и продаж, было проведено модульное тестирование с целью проверки корректности реализации всех ключевых методов. Тестирование выполнялось с использованием фреймворка **MSTest** в среде Visual Studio 2022.

Цели и задачи тестирования

˜Проверить правильность выполнения операций добавления, удаления, обновления и получения данных из базы.

˜Убедиться, что расчёт скидки для партнёра выполняется в соответствии с заданными правилами.

˜Обеспечить изоляцию тестов от реальных данных, чтобы они не влияли на рабочую базу.

Подход к тестированию

Для каждого тестового метода создавался отдельный контекст базы данных (ApplicationContext) и запускалась транзакция. После выполнения проверок транзакция откатывалась (Rollback), что гарантирует неизменность исходных данных. Такой подход позволяет выполнять тесты многократно без риска загрязнения базы.

Описание тестовых сценариев:

Класс GulyaevServiceTests содержит следующие тесты:

| Метод теста | Проверяемое действие |
|---|---|
| AddPartner_AddsPartnerToDatabase | Добавление нового партнёра в таблицу Partners. |
| DeletePartner_RemovesPartnerFromDatabase | Удаление существующего партнёра. |

| Метод теста | Проверяемое действие |
| --- | --- |
| UpdatePartner_UpdatesPartnerInDatabase | Обновление информации о партнёре (изменение наименования). |
| SaveChanges_PersistsPendingChanges | Сохранение изменений через метод SaveChanges сервиса. |
| GetAllPartnerInfo_ReturnsInsertedPartner | Получение списка всех партнёров, включая только что добавленного. |
| GetPartnerInfo_ReturnsCorrectPartner | Получение информации о конкретном партнёре по идентификатору. |
| GetAllSalesInfo_ReturnsSalesForPartner | Получение истории продаж для заданного партнёра. |
| GetProductInfo_ReturnsProductNames | Получение перечня наименований продукции. |
| GetDiscount_ReturnsExpectedValues | Проверка расчёта скидки на основе объёма продаж (0%, 5%, 10%, 15%). |
| AddSale_AddsPartnerProduct | Добавление записи о реализации продукции. |
| UpdateSale_UpdatesPartnerProduct | Обновление количества проданной продукции. |

| Метод теста | Проверяемое действие |
| --- | --- |
| DeleteSale_RemovesPartnerProduct | Удаление записи о продаже. |

Таблица 1 – Перечень модульных тестов и проверяемых действий

Каждый тест включает в себя подготовку тестовых данных, выполнение соответствующего метода библиотеки и проверку ожидаемого результата с помощью методов класса Assert.

Пример теста:

Ниже приведён фрагмент кода одного из тестов (полный листинг представлен в приложении):

csharp

```csharp
[TestMethod]
public void AddPartner_AddsPartnerToDatabase()
{
   using var context = CreateContext();
   using var transaction = context.Database.BeginTransaction();
   var service = new Service();
   var partner = new Partner { PartnerName = "Test Partner", PartnerType = "TP", ... };

   service.AddPartner(context, partner);

   var exists = context.Partners.Any(p => p.Id == partner.Id);
   Assert.IsTrue(exists);
   transaction.Rollback();
}
```

Результаты тестирования

Все описанные тесты успешно проходят. Это подтверждает:

- корректную реализацию CRUD-операций в библиотеке;
- правильность алгоритма расчёта скидки;
- стабильность взаимодействия с базой данных через Entity Framework Core.

Модульное тестирование позволило своевременно выявить и устранить потенциальные ошибки, а также подтвердить соответствие библиотеки предъявленным функциональным требованиям. Все тесты оформлены в соответствии

с лучшими практиками (изоляция через транзакции, осмысленные имена методов, однозначные проверки). Это обеспечивает высокое качество разработанного программного модуля и упрощает его дальнейшее сопровождение.

# Заключение

В ходе выполнения данной работы была разработана и реализована база данных, предназначенная для хранения и обработки информации о партнёрах, типах продукции, товарах и операциях взаимодействия между ними. Разработка базы данных выполнялась с использованием системы управления базами данных PostgreSQL.

На начальном этапе работы была создана роль пользователя с необходимыми правами доступа, а также создана база данных, предназначенная для хранения всей информации проекта. После этого было выполнено подключение к базе данных и создана отдельная схема, которая используется для логической организации объектов базы данных.

В рамках проектирования структуры базы данных были разработаны основные таблицы системы. Таблица partners предназначена для хранения информации о партнёрах, включая название организации, контактные данные, адрес и рейтинг. Таблица product_type используется для хранения типов продукции. Таблица products содержит информацию о конкретных товарах, включая их название, тип продукции, артикул и стоимость. Таблица partner_products предназначена для хранения информации о взаимодействии партнёров с продукцией, включая количество товара и дату продажи.

Между таблицами были реализованы связи с использованием внешних ключей, что обеспечивает целостность данных и позволяет корректно организовать структуру хранения информации. Для повышения производительности выполнения запросов был создан индекс для таблицы partners по полю partner_name, что позволяет ускорить поиск данных по названию партнёра.

Кроме того, в рамках работы было выполнено наполнение базы данных тестовыми данными, что позволило проверить корректность структуры таблиц и связей между ними. Также было выполнено резервное копирование базы данных с использованием встроенной утилиты pg_dump, что обеспечивает возможность восстановления данных в случае возникновения ошибок или сбоев системы.

Для наглядного представления структуры базы данных была построена схема базы данных с использованием инструмента моделирования, на которой отображены все таблицы, их поля, а также связи между ними.

В результате выполнения работы были закреплены практические навыки создания и администрирования баз данных, проектирования структуры таблиц, настройки связей между таблицами, создания индексов, а также выполнения резервного копирования данных. Полученные результаты могут быть использованы в дальнейшем при разработке информационных систем и программных продуктов, использующих базы данных для хранения и обработки информации.

# ПРИЛОЖЕНИЯ

## ПРИЛОЖЕНИЕ 1

### Описание предметной области

Производственная компания «Мастер пол» выпускает напольные покрытия. Компания занимается производством и реализует свою продукцию через партнеров, которые доставляют продукцию компании до конечных потребителей.

С целью оптимизации деятельности компании разрабатывается система, в которой заказчик выделил несколько подсистем:

– продукция;

– склад и материалы;

– производство;

– сотрудники;

– партнеры.

В данной области определены следующие основные составляющие:

Партнеры: тип, наименование компании, юридический адрес, ИНН, ФИО директора, контактные данные (телефон, email), логотип, рейтинг, места продаж, история реализации продукции для последующего расчета скидок.

Партнеры покупают продукцию со скидками, которые зависят от общего объема их продаж. Скидки мотивируют партнеров к большему объему продаж. Партнеры реализуют товары в розничных и оптовых магазинах, интернет-магазинах, другим компаниям по продаже товаров и оказанию услуг.

Взаиморасчеты производственной компании с партнерами осуществляются в рублях.

Менеджеры: поиск и регистрация партнеров, решение об изменении рейтинга партнера, ведение истории изменений рейтинга, прием заявок от партнеров, формирование для них предложения на основании статистики истории продаж, полученной от аналитиков компании.

Заявка: если партнер ранее производил закупку продукции и предоставил данные о ее реализации, то на основании этих данных и текущих объемах готовой

продукции менеджер формирует предложение. Если работа ведется с новым партнером, то менеджер предоставляет каталог готовой продукции и принимает заявку. Затем менеджер должен указать стоимость и дату производства каждой указанной в заявке единицы продукции. После согласования списка продукции с количеством, стоимостью и сроком изготовления заявка считается созданной.

Партнер может отменить заявку до момента внесения предоплаты. Если предоплата не поступает в течение трех дней, то заявка отменяется автоматически, о чем обязательно уведомляется партнер. После поступления предоплаты продукция, указанная в заявке и отсутствующая на складе в нужном количестве, запускается в производство. По готовности всей продукции менеджер должен предложить организовать доставку. В момент получения продукции (путем доставки или самовывоза) партнер производит полную оплату. После проверки количества и качества продукции партнером заявка считается выполненной.

Сотрудники: ФИО, дата рождения, паспортные данные, банковские реквизиты, наличие семьи, состояние здоровья.

Кадры: допуск к работе с конкретным оборудованием. Мастер производства занимается технологией производства, контролем качества готовой продукции, заказом материалов у поставщиков.

Доступ: двери открываются при использовании карты сотрудника, система турникетов передает данные о перемещении сотрудников в приложение для формирования аналитиками статистики по сотрудникам.

Материалы (сырье): тип, наименование, поставщик, количество в упаковке, единица измерения, описание, изображение, стоимость, количество на складе, минимальное допустимое количество, история изменений количества материалов на складе.

Склад: регистрация поступлений материалов, резервирование материалов для производства, отпуск и списание материалов со склада в производство, поступление и выбытие готовой продукции, получение информации о текущих остатках и движении товарно-материальных ценностей по складу.

Поставщики: тип, наименование, ИНН, история поставок материалов. Менеджеры контролируют список поставщиков, аналитики формируют информацию

о качестве поставляемых материалов для принятия решения о прекращении работы с определенными компаниями.

Продукция: артикул, тип, наименование, описание, изображение, минимальная стоимость для партнера, размер упаковки (длина, ширина, высота), вес без упаковки, вес с упаковкой, сертификат качества (скан документа), номер стандарта, история изменений минимальной стоимости для партнера, время изготовления, себестоимость, номер цеха, количество человек на производстве, необходимые материалы для производства.